

Game Registry: Planning Document

How will it work?

*Note that this is an initial plan & outline, and does not necessarily match the final product of our class diagram, Use Case Diagram, and other supporting documents.

Scope & Overview

This document outlines the **Game Registry** system for the platform. The game registry system manages the relationship between the server-side database and the client-side game execution. The registry ensures that game information is consistent across all clients and facilitates the launch process and integration with the **leaderboard** system.

Structure (based on class diagram)

- **GameRegistry:** Centralizes game management. Uses `listAll()` to populate the UI and `findById(id)` to retrieve specific game data for launching games
- **Game & GameDTO:** Data is handled via `GameDTO` for efficient network transmissions of titles and IDs. The full `Game` class contains the logic for `launch()` and `getMetadata()`.
- **LaunchConfigs:** Manages execution variables such as `gameUrl`, `fullscreen`, and `openInNewTab`.
- **Tagging:** Enables categorization (like “Multiplayer”) via the `Tag` class for filtering and match making.

Functional Use Cases

UC-01 & UC-02: Administrative Management

- **Add New Game:** The admin inputs the Title, Description, and Launch command. The system automatically generates a unique ID: String, and persists the object to the server-side database. A confirmation is required to complete the process.
- **Remove Game:** Admins can delete games via ID or Title. To prevent accidental data loss, the system includes a mandatory Confirmation step.

UC-03: Library Visualization

- **View Games:** Pulls a list of `GameDTO` objects to display available titles to the user. Upon selecting a game, the system retrieves full details (Description, ID, etc.) from the database for a more in-depth-view.

UC-04 & UC-05: Game Execution Flow

- **Launch Handshake:** When a user selects “Play”, the system retrieves the specific LaunchConfigs and start command
- **System Execution:** The Registry triggers the RunGame sub-process (UC-05), executing the retrieved command and recording the status (Success/Fail) to inform the user if the game fails to launch.

Network

- **Connection Protocol:** Clients initiate a TCP/IP connection to the server’s IP to request game updates
- **Efficiency:** To avoid handling connections indefinitely, the GameRegistry fetches the necessary GameDTO list and immediately breaks the connection until a “Launch” or “Details” request is made
- **Security:** Game IDs are tied to the server database, ensuring that even if a user edits local files, they cannot spoof a game's identity when reporting scores to the leaderboard.

Questions For Other Teams

- **Leaderboard Team:** Does the execution status recorded in UC-05 need to be sent to the leaderboard team to track “Games Played” statistics?
- **Database Team / Leaderboard:** Would it be possible to implement a “Last Launched” timestamp in the Game class to help users sort their library? It could make it more user-friendly.
- **Integration & Release:** How can we best integrate our work with the database team and the leaderboard team
- **General:** What types of games are we implementing? (Single-player vs bot? Real-time multiplayer?)
- **General:** Are we restricting the number of players per game to 2 people?
- **Leaderboard:** Are scores compared after single-player sessions, or are they generated from 2-player matches?